



Java

Clase 05 | Programación Orientada a Objetos

Índice

Clase 05

Clase 5 | Programación Orientada a Objetos (POO)

- Paradigmas de programación: estructurada vs. POO.
- Clases, objetos y atributos.
- Creación de clases y objetos en Java.
- Métodos de instancia y de clase.
- Constructores y métodos especiales.

Clase 06

Clase 6 | Encapsulamiento y colaboración entre clases

- Encapsulamiento y visibilidad (public, private, protected).
- Métodos de acceso (getters y setters).
- Colaboración entre clases.
- Objetos dentro de objetos.

Variables de clase (static).

Objetivos de la Clase



Comprender Paradigmas

Diferenciar entre programación estructurada y orientada a objetos.



Clases y Objetos

Entender qué son y cómo representan entidades del mundo real.



Métodos

Distinguir entre métodos de instancia y de clase.



Constructores

Conocer su función en la inicialización de objetos.

Introducción a la Programación Orientada a Objetos (POO)

POO

La programación orientada a objetos (POO) es un paradigma de programación que utiliza objetos como unidades fundamentales de un programa. Los objetos son entidades que encapsulan datos y comportamiento, representando conceptos del mundo real. La POO facilita la organización y el mantenimiento del código, promoviendo la reutilización y la modularidad.



Características de la programación orientada a objetos



Encapsulamiento

Cada objeto controla su estado interno, exponiendo solo las operaciones necesarias.



Abstracción

Ocultar la complejidad interna de los objetos, presentando una interfaz sencilla.



Herencia

Permite crear nuevas clases basadas en otras existentes.



Polimorfismo

Objetos de distintas clases pueden ser tratados de la misma manera si comparten una interfaz común.

Programación Estructurada vs. POO

Estructurada

Divide el programa en funciones y procedimientos secuenciales. Los datos y las operaciones están separados, siguiendo un flujo lineal de ejecución. Se basa en tres conceptos: secuencia, selección y repetición.

POO

Organiza el código en objetos que encapsulan datos y comportamientos relacionados. Permite modelar el software como interacciones entre entidades del mundo real, facilitando la reutilización y el mantenimiento del código.

Clases y Objetos: La Plantilla y su Instancia

En POO, las clases son como plantillas o blueprints que definen la estructura y comportamiento de los objetos. Los objetos son instancias concretas de una clase, es decir, copias de la plantilla que toman vida propia. Imagina una clase como un molde para galletas: define la forma y los ingredientes. Cada galleta que horneas es un objeto, una instancia de ese molde. Las clases proporcionan un esquema para crear objetos con propiedades y acciones definidas.



Clases

Introducción a las Clases en Java

Las clases son la base de la programación orientada a objetos (POO) en Java. Actúan como plantillas que definen la estructura y comportamiento de los objetos.

1 Plantillas Reutilizables

Las clases permiten crear múltiples objetos con las mismas características, promoviendo la reutilización del código.

2 Abstracción

Oculto la complejidad interna de los objetos, presentando una interfaz sencilla para su uso.

3 Encapsulación

Protege los datos internos de los objetos, controlando el acceso a través de métodos.

Clases, Objetos y Atributos

Clase

Es el modelo o plantilla que define la estructura básica de algo. Como un plano arquitectónico, establece las características y comportamientos que tendrán todos los objetos creados a partir de ella.

Objeto

Es una instancia específica y funcional de una clase. Si la clase es el plano de una casa, el objeto es la casa construida con todas sus características particulares. Cada objeto es único aunque comparta el mismo diseño base.

Atributos

Son las propiedades o características específicas que describen el estado de un objeto. Como el color, tamaño o precio de un producto. Los atributos almacenan los valores que hacen único a cada objeto.

Atributos

Introducción a los Atributos en Java



¿Qué son los Atributos?

Los atributos son variables que se declaran dentro de una clase y representan las características o propiedades que tendrán los objetos. Son la forma de almacenar datos en la programación orientada a objetos.



¿Para qué sirven?

Los atributos permiten definir el estado y las características específicas de cada objeto. Por ejemplo, un objeto **Coche** puede tener atributos como **color**, **modelo** y **velocidad**.



Declaración en Java

En Java, los atributos se declaran dentro de la clase usando un tipo de dato (**int**, **String**, **boolean**, etc.) y un nombre descriptivo. Por ejemplo: **private String nombre;**

Ejemplo: Clase Producto con sus atributos

```
public class Producto {  
    String nombre;  
    double precio;  
    int cantidadEnStock;  
}
```

Esta clase Producto actúa como una plantilla a partir de la cual podemos crear múltiples objetos o instancias. Por ejemplo, podríamos crear diferentes productos como "Laptop", "Teléfono" o "Tablet", cada uno con sus propios valores específicos de nombre, precio y cantidad en stock. Cada objeto creado será una instancia única de la clase Producto, con su propio espacio en memoria y valores independientes para sus atributos.

Resultado: Creando Objetos Producto

```
// Creando objetos Producto
Producto producto = new Producto();
producto.nombre = "Producto Premium";
producto.precio = 250.0;
producto.cantidadEnStock = 100;
```

Explicación del Código

- Cada producto es una instancia independiente de la clase Producto
- Cada objeto tiene sus propios valores para los atributos definidos en la clase
- Los objetos pueden ser manipulados de forma individual
- Podemos acceder a sus atributos usando la notación punto (objeto.atributo)

Esta implementación nos permite manejar cada producto como una entidad completa, con todos sus atributos y comportamientos asociados, en lugar de datos dispersos en diferentes estructuras.

Problema: Creación de la clase Café

La cafetería Sibelius necesita implementar un sistema de gestión de productos. Para ello, se requiere crear una clase llamada "Cafe" (sin tilde, siguiendo las convenciones de Java) que servirá como plantilla para crear diferentes tipos de café en el sistema. Esta clase debe contener los atributos necesarios para gestionar las ventas y el inventario.

```
public class Cafe {
    // Atributos
    private String nombre;
    private double precio;
    private String tamaño;
    private boolean conLeche;
    private int cantidadStock;
```

```
    // Constructor
    public Cafe(String nombre, double precio,
String tamaño, boolean conLeche, int
cantidadStock) {
        this.nombre = nombre;
        this.precio = precio;
        this.tamaño = tamaño;
        this.conLeche = conLeche;
        this.cantidadStock = cantidadStock;
    }
}
```


Metodos de clase

Introducción a los metodos dentro de las clases

Las clases, además de parámetros tienen métodos. Los métodos son componentes fundamentales que dan vida a nuestras clases en Java, definiendo cómo se comportan los objetos.



Comportamiento de la Clase

Los métodos definen las acciones que puede realizar un objeto, como `calcularDescuento()` o `actualizarStock()`.



Entrada y Salida

Pueden recibir parámetros y devolver resultados, permitiendo una comunicación efectiva entre objetos.



Encapsulamiento

Se definen dentro de la clase y se acceden a través de los objetos utilizando la notación punto: `producto.calcularTotal()`

Ejemplo: Métodos en Producto

```
public class Producto {  
    // ... atributos ...  
  
    public void descontarStock(int cantidad) {  
        this.cantidadEnStock -= cantidad;  
    }  
  
    public static double calcularImpuesto(double precio) {  
        return precio * 0.21; // 21% de impuestos  
    }  
}
```

● Problema: Métodos de la clase café

La cafetería Sibelius necesita implementar un método en su clase Cafe que calcule el precio final del café según si lleva leche o no. Si el café lleva leche, se debe agregar un cargo adicional de \$50 al precio base. Implementá el método calcularPrecio() que realice este cálculo.

```
public class Cafe {  
    // Atributos  
    private String nombre;  
    private double precio;  
    private String tamaño;  
    private boolean conLeche;  
    private int cantidadStock;
```

```
// Constructor  
    public Cafe(String nombre, double precio, String tamaño,  
boolean conLeche, int cantidadStock) {  
        this.nombre = nombre;  
        this.precio = precio;  
        this.tamaño = tamaño;  
        this.conLeche = conLeche;  
        this.cantidadStock = cantidadStock;  
    } // Método para calcular precio final  
    public double calcularPrecio() {  
        double precioFinal = this.precio;  
        if (this.conLeche) {  
            precioFinal += 50; // Cargo adicional por la leche  
        }  
        return precioFinal;  
    }  
}
```

Constructores en Java

Introducción a los Constructores en Java

En Java, los constructores son métodos especiales que se utilizan para inicializar objetos cuando se crean. Se ejecutan automáticamente al instanciar una clase. Los constructores tienen el mismo nombre que la clase y no tienen tipo de retorno. Su principal función es establecer los valores iniciales de los atributos de un objeto.

Si no se define un constructor, Java genera un constructor por defecto que no inicializa los atributos.

Los constructores son esenciales para garantizar que los objetos se creen en un estado válido y consistente.

Ejemplo: Constructor en Clase Producto:

```
public class Producto {  
    private String nombre;  
    private double precio;  
  
    public Producto(String nombre, double precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
}
```

Problema: Constructores

La cafetería Sibelius necesita crear una clase Cafe para gestionar sus productos. La clase debe tener los siguientes atributos: nombre, precio, tamaño, si lleva leche o no, y cantidad en stock. Implementará la clase Cafe con un constructor que inicialice todos estos atributos y un método calcularPrecio() que agregue un cargo adicional de \$50 si el café lleva leche.

```
public class Cafe {  
    // Atributos  
    private String nombre;  
    private double precio;  
    private String tamaño;  
    private boolean conLeche;  
    private int cantidadStock;
```

```
    // Constructor  
    public Cafe(String nombre, double precio, String  
tamaño, boolean conLeche, int cantidadStock) {  
        this.nombre = nombre;  
        this.precio = precio;  
        this.tamaño = tamaño;  
        this.conLeche = conLeche;  
        this.cantidadStock = cantidadStock;  
    } // Método para calcular precio final  
    public double calcularPrecio() {  
        double precioFinal = this.precio;  
        if (this.conLeche) {  
            precioFinal += 50; // Cargo adicional por la leche  
        }  
        return precioFinal;  
    }  
}
```


Recursos Adicionales

Documentación oficial

Consulta la documentación de Java para profundizar en los conceptos.



docs.oracle.com



Class (Java Platform SE 8)

Class has no public constructor.
Instead Class objects are
constructed automatically by th...

Guía de métodos

Revisa la guía de Oracle sobre métodos en Java.

<https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html>

Videos recomendados

Mira "Organización del Código con Métodos" en YouTube.

https://youtube.com/playlist?list=PL954bYq0HsCXlwPGM2k2nxRVztH0wpPqc&si=GfIFagrxE_BjMaw4

Ejercicios prácticos

Situación inicial en TechLab.



Silvia, la Product Owner, observa que el código del proyecto de e-commerce comienza a ser difícil de mantener: tenemos datos de productos, clientes, descuentos y otras lógicas dispersas en funciones y variables globales. Matías y Sabrina notan la necesidad de un cambio de paradigma. En lugar de manipular datos de manera aislada, quieren "dar vida" a estos datos creando entidades llamadas objetos, cada uno con atributos (datos) y comportamientos (métodos).

En suma, el equipo se prepara para migrar hacia un enfoque orientado a objetos. Esto facilitará la representación del dominio (productos, clientes, pedidos) de forma más natural, haciendo el código más legible y flexible ante futuros cambios.

Ejercicio práctico

Optativo | No entregable

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

Creación de clases y objetos

- Crea una clase Cliente con atributos nombre y email.
- Instanciá un objeto Cliente y asigne valores a sus atributos.

Métodos

- En la clase Producto, agrega un método mostrarInformacion() que imprima nombre, precio y stock.

Ejercicio práctico

Optativo | No entregable

Mas métodos en la misma clase

- Agregá un método en Producto que calcule un descuento general para todos los productos de un 10%.
- Probalo con distintos precios.

Constructores

- Creá un constructor para Cliente que reciba el nombre y email.
- Crea varios clientes con este constructor y mostrálos por pantalla.